Polish-Japanese Academy of Information Technology

Bachelor's engineering thesis

Short-Term Forecasting of Martian Surface Weather Using Machine Learning

PRO3, Task 06: Chapter 6, Models and methods

Uladzislau Staliarou (s29905)

November 2026

6 Models and methods

The preceding chapters prepared the station records (Chapters 3 and 4) and described the software that processes them (Chapter 5). This chapter defines the forecasting problem and the four model classes that address it, two reference forecasts and two learned models, together with the hypotheses, the training procedures and the evaluation protocol whose results Chapter 7 reports. Section 6.1 formulates the problem, states the loss function and the metrics and fixes the hypotheses and success criteria. Section 6.2 describes how the preprocessed data enter each model and how the training, validation and test periods were formed. Sections 6.3 to 6.5 describe the reference forecasts, the gradient-boosted decision trees (GBDT) and the temporal convolutional network (TCN), each with its rationale, structure, hyperparameters and training setup, and Section 6.6 the design of the cross-site experiments. Section 6.7 specifies the evaluation and Section 6.8 the diagnostics with which the models are interpreted. All data of the project are tabular time series (Chapter 3), so one structure serves every model, and methods specific to text, images, audio or video are not needed.

6.1 Problem formulation

6.1.1 Forecasting task

Let $v(t)$ denote one of the three target variables, the air temperature, the pressure or the horizontal wind speed, at the issue time t, the time at which a forecast is made. The task is to estimate the value of the same variable H hours later,

$$y(t) = v(t + H), \qquad H \in \{1, 6, 24, 48\} \text{ h}, \tag{6.1}$$

from the measurements of the station up to and including t. The task is a supervised regression with a continuous target in physical units (kelvin, pascal, meters per second). The four horizons probe different parts of the problem: one hour tests how far the current state persists, six hours crosses the steepest part of the diurnal ramp, 24 hours tests whether a model exploits the daily period, and 48 hours reaches beyond the sol of history that the models see.

Each horizon is forecast directly. One model is fitted per target and horizon, twelve per model class, instead of iterating a one-step model forward, a recursive strategy whose errors accumulate from step to step[1]. The direct strategy also places the reference forecasts, which involve no iteration, on the same footing as the learned models.

The experiments use the two time grids of Chapter 4. The single-site experiments of RQ1 and RQ2 run on the 5-minute grid of MEDA, with H in Earth hours, that is, 12, 72, 288 and 576

[1] Cf. H. Hewamalage, C. Bergmeir, and K. Bandara. "Recurrent Neural Networks for Time Series Forecasting: Current status and future directions". In: *International Journal of Forecasting* 37.1 (2021), pp. 388–427.

steps of the grid, and with the absolute value as the target. The cross-site experiments of RQ3 run on the hourly grid of local mean solar time (LMST) shared by the three stations, with H in Mars hours of 3,699 s and with the change over the horizon as the target,

$$y(t) = v(t + H) - v(t), \tag{6.2}$$

whose forecast is scored after adding $v(t)$ back, so that all errors are in the units of the variable. The hourly grid aligns the stations by local time, and the change removes the level of each site (Chapter 4). The inputs are, in every experiment, the station's own measurements up to t and quantities known at t from the calendar and the solar geometry; no physical model output and no data of a later time enter.

6.1.2 Loss function and metrics

Both learned models minimize the mean squared error (MSE) of their forecasts on the training period: the GBDT through the squared-error objective of its library and the TCN through the MSE of z-scored targets (Section 6.5). The squared error is minimized by the conditional mean of the target, which is what a point forecast should estimate. Accuracy is reported in the same units, as the root mean square error (RMSE) and the mean absolute error (MAE) over the n scored issue times of a target and horizon,

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}, \qquad \mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|\,. \tag{6.3}$$

The RMSE is the primary metric because it penalizes large errors, which on Mars occur at the diurnal ramps and during dust events, where a forecast matters most, and because it corresponds to the training loss. The MAE is reported as a robustness check that single large errors affect less.

Accuracy relative to a reference forecast is expressed by the skill score

$$S = 1 - \frac{\mathrm{RMSE}_{\mathrm{model}}}{\mathrm{RMSE}_{\mathrm{ref}}}, \tag{6.4}$$

which is 0 for a model as accurate as the reference, 1 for a perfect model and negative for a model worse than the reference. The score is formed from RMSEs so that it is in the same proportion as the reported errors. It is related to the mean-square skill score of Murphy[2], $1 - \mathrm{MSE}_{\mathrm{model}}/\mathrm{MSE}_{\mathrm{ref}}$, by $\mathrm{SS}_{\mathrm{MSE}} = 1 - (1 - S)^2$, so that $S = 0.5$ corresponds to $\mathrm{SS}_{\mathrm{MSE}} = 0.75$. Persistence is the reference named in the success criteria below; the skill against the more accurate of persistence and the typical-day forecast is reported as well, as Murphy recommended, because either reference alone can flatter a model at some horizons[3]. Being dimensionless, the skill score also allows the

[2] A. H. Murphy. "Skill Scores Based on the Mean Square Error and Their Relationships to the Correlation Coefficient". In: *Monthly Weather Review* 116.12 (1988), pp. 2417–2424.

[3] Cf. A. H. Murphy. "Climatology, Persistence, and Their Linear Combination as Standards of Reference in Skill Scores". In: *Weather and Forecasting* 7.4 (1992), pp. 692–698.

three targets, whose units differ by orders of magnitude, to be compared.

6.1.3 Hypotheses and success criteria

The hypotheses were formulated at the start of the project, before any model was trained, and their thresholds were registered as reporting targets rather than as pass-or-fail gates: the measured numbers are reported whether or not the thresholds are met.

H1 A model with non-linear capacity, the GBDT or the TCN, reduces the 24-hour RMSE by at least 20 % relative to persistence on at least two of the three targets at the primary site.

H2 Adding the secondary signals to the inputs reduces the RMSE of a primary target by at least 5 % relative to the same model trained on the primary channels alone, reproducibly at two horizons.

H3 A model trained at one site and evaluated at another has a materially higher error than the model trained at the evaluation site, with a relative RMSE increase of more than 50 % on at least one variable.

Hypothesis H3 is exploratory: a smaller transfer gap would itself be an informative finding. Table 6.1 summarizes how the three experiments operationalize the research questions of Chapter 1, what each varies and holds fixed, and the rule by which each hypothesis is judged. The decision rule of H2 was tightened in October 2026, before the corresponding runs were made, after a single pair of runs proved sensitive to the random seeds of the GBDT (Section 6.7).

Table 6.1: Design of the three experiments

RQ	Varied factor	Held fixed	Comparison	Decision rule
RQ1	model class: persistence, typical day, GBDT, TCN	MEDA 5-minute table, primary inputs, periods, horizons, metrics	each model against both references on identical test rows; skill score	H1: RMSE at 24 h at least 20 % below persistence on two of three targets
RQ2	feature set: primary only; primary and secondary	GBDT settings, periods, horizons; five LightGBM seeds per set	paired RMSE difference per target and horizon on identical rows, with a sol-block interval	H2: a clear gain of at least 5 % at two horizons; clear means all five seed pairs agree in sign and the interval excludes zero
RQ3	training data: each site alone; the two other sites pooled	hourly grid, change targets, GBDT settings, references	cross-site against in-site RMSE on the same test rows; transfer gap with a sol-block interval	H3: gap above 50 % on at least one variable; reported descriptively as a matrix

Source: own elaboration.

6.2 Data representation and periods

6.2.1 Inputs of the models

After the preprocessing of Chapter 4, the data reach the models in three forms (Table 6.2). The GBDT receives, for each issue time, one row of the feature table: a vector of 77 numerical features in the primary-only set and of 132 in the set with secondary signals. Six series, the three targets, the wind components u and v and the deviation of pressure from its seasonal baseline, each contribute their values 5, 60, 360 and 1,440 minutes before t and the mean and standard deviation over windows of 60, 360 and 1,440 minutes ending at t. The current wind components, the current pressure deviation and the seasonal baseline, the LMST hour with its sine and cosine, the seasonal coordinate $\tilde{L}_s$ with its sine and cosine, the solar zenith and azimuth angles of the ancillary product and five flags marking missing sensors complete the vector. The value $v(t)$ of the target itself is not a column of the vector: it enters through its 5-minute lag, through the window means, which include t, and, for pressure and wind, through the deviation and the components. The set with secondary signals adds the current value of the water-vapor mixing ratio, the temperature of the humidity sensor, the two longwave fluxes and the solar radiation, and the same ten lag and window features for each, 55 features in all. The five secondary channels are added as one block, because several of them are closely related (Chapter 4), so that their joint value is tested rather than that of each channel. The features keep their physical units, and missing values are passed to the GBDT as they are.

The TCN receives a tensor of 14 channels by 288 steps, the 24 hours of 5-minute intervals that end at t: the air temperature, the pressure, u and v, the four calendar encodings, the seasonal pressure baseline and the five missingness flags. Every channel except the flags is z-scored with the mean and standard deviation of its observed values at the training issue times, and missing values are set to zero, the training mean, after scaling (Chapter 4). Windows that reach before the first interval of the table are padded with the training mean. The 288 steps cover 97.3 % of a sol; the TCN is used with the primary channels only. For RQ2 the GBDT serves as the fixed model class, for two reasons: LightGBM handles missing values natively, and the humidity variable is present only at night, in 31 % of the intervals, whereas the TCN would receive it mostly as a filled constant; and the GBDT trains in seconds, which made the repetition of both feature sets with five seeds affordable.

The cross-site GBDT receives 15 features of the target variable alone: its changes over 1 to 48 hours, its deviation from and standard deviation over the preceding 6 and 24 hours, and the sine and cosine of the LMST hour and of the solar longitude. A row is used only when all 15 features and the target are present (Chapter 4).

6.2.2 Training, validation and test periods

Every record was divided chronologically into a training, a validation and a test period of 70, 15 and 15 % of its sols (Figure 6.1, Table 6.3). Forecasting models are evaluated out of sample, on

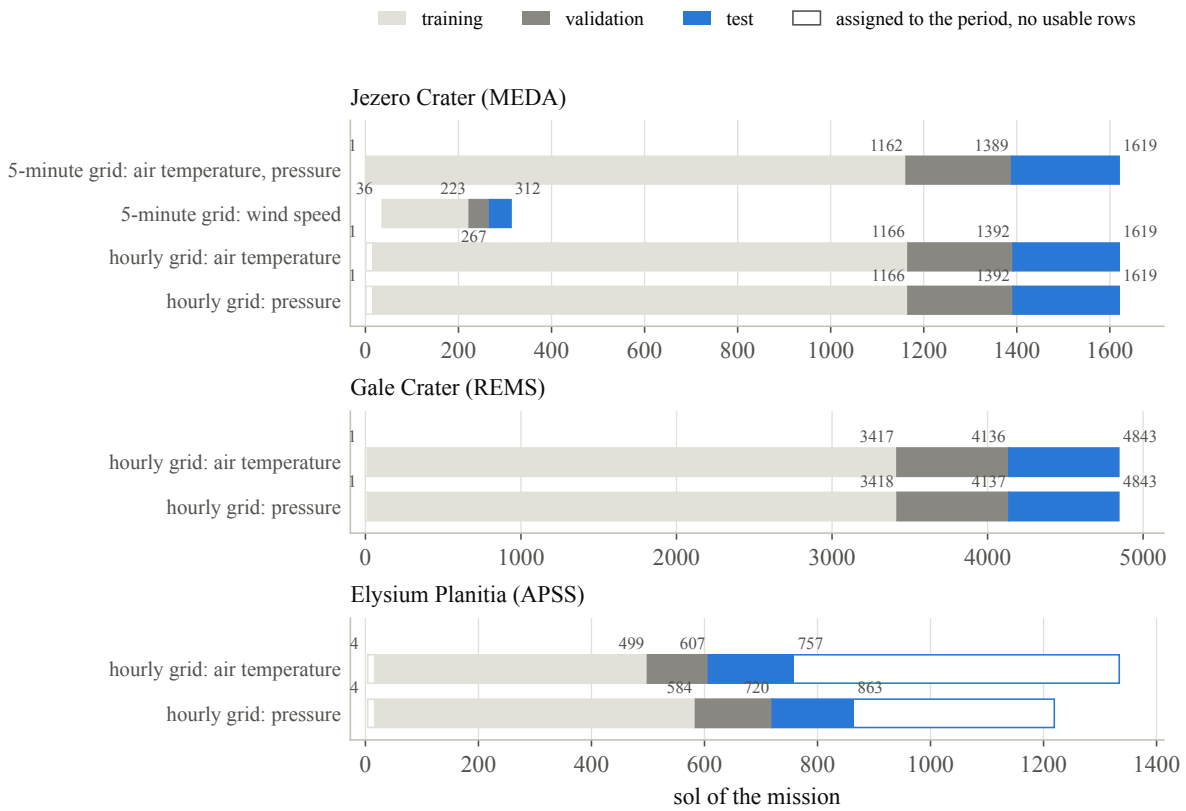
Table 6.2: Inputs of the models after preprocessing

Representation	Group	Members	Count
GBDT, primary feature vector	lags	values 5, 60, 360 and 1,440 min before t of air temperature, pressure, wind speed, u, v and the pressure deviation	24
	windows	mean and standard deviation over 60, 360 and 1,440 min, same six series	36
	current state	u, v, pressure deviation, seasonal pressure baseline	4
	time of sol	LMST hour, its sine and cosine	3
	season	$\tilde{L}_s$, its sine and cosine	3
	solar geometry	zenith and azimuth angle of the Sun	2
	missingness flags	pressure, wind, humidity, thermal infrared and radiation sensors	5
	total		77
GBDT, with secondary signals	secondary channels	current value, four lags and six window statistics of the mixing ratio, the humidity-sensor temperature, both longwave fluxes (the upward one capped) and the solar radiation	55
	total		132
TCN, window of 288 steps	primary channels	air temperature, pressure, u, v	4
	calendar	sine and cosine of the LMST hour and of $\tilde{L}_s$	4
	baseline	seasonal pressure baseline	1
	missingness flags	as above	5
	total channels		14
Cross-site GBDT, hourly grid	changes	$v(t) - v(t-k)$ for $k = 1, 2, 3, 6, 12, 24, 48$ h	7
	windows	deviation from the mean and standard deviation over 6 and 24 h	4
	calendar	sine and cosine of the LMST hour and of L_s	4
	total		15

u, v: wind components; $\tilde{L}_s$: approximate solar longitude from the sol number; L_s: solar longitude from the Mars24 algorithm (Chapter 4).

Source: own elaboration based on the project code.

Figure 6.1: Training, validation and test periods of the experiments



Source: own elaboration based on the feature table and the hourly tables (Chapter 4).

data that follow the training period[4]; random cross-validation can be valid for stationary series, but the Martian records, with their seasonal cycle, the drift of a climbing rover and sensor failures, are not stationary (Chapter 4), and a random division would place sols of the test period between training sols[5]. The division is made at sol boundaries, so that every diurnal cycle lies within one period.

On the 5-minute grid, the 1,473 sols of the MEDA table that carry a sol number were ordered and divided: sols 1–1161 form the training period, 1162–1388 the validation period and 1389–1619 the test period; the 5.1 % of intervals without a sol number are excluded. Because derived wind exists only for sols 36–312, wind has its own division of that window: sols 36–222, 223–266 and 267–312. The validation period of the main division coincides with the rover's climb to the crater rim and the test period lies on the rim (Chapter 4), a circumstance that the models face and that Chapter 7 discusses. On the hourly grid, each site and target was divided separately. The boundaries were placed over the sols with at least twelve hourly values of the target, so that the sparsely sampled sols late in the InSight mission do not fill its test period, and rows whose target time falls in a different period from their issue time were removed. For InSight

[4]Cf. L. J. Tashman. "Out-of-sample tests of forecasting accuracy: an analysis and review". In: *International Journal of Forecasting* 16.4 (2000), pp. 437–450.

[5]Cf. C. Bergmeir and J. M. Benítez. "On the use of cross-validation for time series predictor evaluation". In: *Information Sciences* 191 (2012), pp. 192–213.

Table 6.3: Periods of the experiments and number of rows at a horizon of one hour

	Training		Validation		Test	
Grid, station, target	Sols	Rows	Sols	Rows	Sols	Rows
5-min, MEDA, air temperature	1–1161	91,913	1162–1388	22,022	1389–1619	35,496
5-min, MEDA, pressure	1–1161	90,870	1162–1388	21,946	1389–1619	35,410
5-min, MEDA, wind speed	36–222	6,182	223–266	1,882	267–312	2,027
hourly, MEDA, air temperature	1–1165	18,600	1166–1391	4,472	1392–1619	4,774
hourly, MEDA, pressure	1–1165	18,617	1166–1391	4,491	1392–1619	4,774
hourly, REMS, air temperature	1–3416	74,326	3417–4135	15,835	4136–4843	14,534
hourly, REMS, pressure	1–3417	72,482	3418–4136	15,455	4137–4843	14,256
hourly, APSS, air temperature	4–498	10,293	499–606	2,272	607–757[a]	1,652
hourly, APSS, pressure	4–583	11,978	584–719	2,401	720–863[a]	1,499

Rows: issue times with the inputs and a measured target at $H = 1$ h. [a] The test period extends to the end of the record (sols 1333 and 1218) but holds usable rows only up to the sol given.

Source: own elaboration based on the feature table and the hourly tables (Chapter 4).

the test period therefore extends to the end of the record, but usable rows end at sol 757 for air temperature and sol 863 for pressure.

An issue time enters a period's rows when the variable is present at t and measured at $t + H$; on the 5-minute grid the GBDT accepts missing inputs, the TCN receives them as the training mean, and on the hourly grid all inputs must be present. Because MEDA samples some hours of a sol fully and others only for a few minutes (Chapter 3), the row counts depend on the horizon: air temperature has 91,913 training rows at one hour, 121,795 at six hours and 84,294 at 48 hours. Wind has between 6,038 and 9,688 training rows. The validation period served only for early stopping of the two learned models and for the selection of the TCN configuration, and the test period was used once per model, feature set and seed.

6.3 Reference forecasts

Two reference forecasts are first-class results of the thesis, because on Mars either of them can be hard to beat and because the closest precedent reported none (Chapter 2). *Persistence* forecasts the current value, $\hat{y}(t) = v(t)$; on the hourly grid, where the target is a change, it forecasts a change of zero. It has no parameters and needs no training. It is strong where the state changes slowly, at one hour, and where it repeats, at 24 and 48 hours, and weak at six hours, when the forecast time lies on the opposite side of a diurnal ramp.

The *typical day*, a seasonal-naive forecast, is the diurnal climatology of the training period. The measured training values of the target were averaged in 48 bins of 30 minutes of LMST, and the forecast for the issue time t is the mean of the bin that contains the local time of the target, $\mathrm{LMST}(t) + H \cdot 3600/3699$ h modulo 24 h, since one Earth hour advances the local time by 0.973 Mars hours. A bin without training values would be interpolated from its neighbors around the sol. For pressure the climatology is fitted on the deviation from the seasonal baseline

and the baseline at the seasonal coordinate of $t + H$ is added back (Chapter 4). The bin width of 30 minutes was set before the experiments and not tuned. On the hourly grid the corresponding reference is the mean change of the target per issue hour in the training rows. The two references bracket what a learned model must beat: persistence knows the current state and nothing of the cycle, the typical day knows the cycle and nothing of the current state[6]. One defect of the typical day was found in October 2026 and corrected before the reported runs: the climatology had been read at the local time of the issue time instead of the target time, which made the forecast appear to lose to persistence at every horizon for air temperature. An automated test now checks the alignment.

6.4 Gradient-boosted decision trees

6.4.1 Rationale and structure

The GBDT is the classical machine-learning model of the thesis and the model whose inputs are varied in RQ2 and whose training site is varied in RQ3. The reasons for the choice follow from Chapter 2: tree ensembles are the most accurate models on tabular data of moderate size in benchmark comparisons[7], LightGBM in particular was used by practically all leading teams of the M5 forecasting competition[8], it handles missing values natively, which matters for a table in which the humidity, radiation and wind sensors are often silent, and it trains in seconds, which made the five-seed repetition of RQ2 and the 48 models of RQ3 affordable. Its inputs stay in physical units, so its split thresholds and feature importances are readable in kelvin, pascal and meters per second (Chapter 4).

Gradient boosting builds an additive model stage by stage: each new regression tree is fitted to the negative gradient of the loss of the current ensemble, which for the squared error is the vector of residuals, and is added with a small step, the learning rate[9]. LightGBM implements this procedure with features bucketed into histograms[10] and with trees grown leaf by leaf, splitting the leaf that reduces the loss most[11]. Rows with a missing value of the split feature are sent to the

[6]Cf. Murphy, *op. cit.*; R. J. Hyndman and G. Athanasopoulos. *Forecasting: Principles and Practice*. 3rd ed. Melbourne: OTexts, 2021. URL: https://otexts.com/fpp3/ (accessed 2026-10-07).

[7]Cf. L. Grinsztajn, E. Oyallon, and G. Varoquaux. "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" In: *Advances in Neural Information Processing Systems 35 (Datasets and Benchmarks Track)*. 2022, pp. 507–520.

[8]Cf. S. Makridakis, E. Spiliotis, and V. Assimakopoulos. "M5 accuracy competition: Results, findings, and conclusions". In: *International Journal of Forecasting* 38.4 (2022), pp. 1346–1364.

[9]J. H. Friedman. "Greedy function approximation: A gradient boosting machine". In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232.

[10]G. Ke et al. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017, pp. 3146–3154. URL: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html (accessed 2026-10-07).

[11]Cf. Microsoft Corporation. *Parameters*. LightGBM 4.3.0 documentation. 2024. URL: https://lightgbm.readthedocs.io/en/v4.3.0/Parameters.html (accessed 2026-10-28).

branch that reduces the loss more, a direction learned at each split, as introduced by XGBoost[12]. One booster is trained per target and horizon, twelve per feature set and seed; in the cross-site experiments, six boosters per target and horizon, 48 in all.

6.4.2 Hyperparameters and training setup

Table 6.4 lists the hyperparameters. They were set once, before training, from common practice for boosted trees: a small learning rate with many rounds and early stopping, bounds on the depth and the number of leaves that limit the order of interactions a tree can represent, a minimum leaf size and an L2 penalty that regularize the leaf values, and the sampling of rows and features that decorrelates the trees[13]. No search over these values was made; the validation period served only to stop the boosting. The consequence is that the comparison between the GBDT and the TCN in Chapter 7 is one between a model with fixed settings and a model with a small tuning budget, and that the sensitivity of the GBDT results to its settings is not measured.

Table 6.4: Hyperparameters of the GBDT

Parameter	Value	Role
objective	squared error	the loss of Section 6.1.2
learning rate	0.05	step size of each tree; small values need more rounds and generalize better
boosting rounds	up to 2,000	stopped when the validation RMSE has not improved for 100 rounds; the best round is kept
maximum depth	8	bounds the order of interactions in a tree
maximum leaves	63	bounds the size of a leaf-wise grown tree
minimum rows per leaf	50	prevents leaves fitted to a few rows
L2 penalty on leaf values	1.0	shrinks the leaf values
feature fraction	0.8	features sampled per tree
row fraction	0.8, every 5 rounds	rows sampled for the trees
histogram construction	deterministic, row-wise	reproducible results across runs and machines
random seeds	library defaults; 1–5 in the seed check	row and feature sampling
threads	all processor cores	

Source: own elaboration based on the experiment configuration.

Training proceeds on the processor. Each booster is fitted on the training rows of its target

[12]Cf. T. Chen and C. Guestrin. "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.

[13]Cf. T. Hastie, R. Tibshirani, and J. Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer Series in Statistics. New York: Springer, 2009; Microsoft Corporation, *op. cit.*

and horizon, its validation RMSE is computed after every round, and the round with the lowest validation RMSE is kept once 100 rounds have passed without improvement. In the run on the primary feature set, early stopping kept between 64 and 163 rounds for eleven of the twelve boosters and 1,503 rounds for pressure at one hour; in the cross-site experiments, between 13 and 2,000 rounds, and four pressure boosters at one and six hours were still improving at the limit of 2,000 rounds. At the kept round the training RMSE lies well below the validation RMSE: 1.5–1.9 K against 2.4–3.0 K for air temperature, 0.7–3.9 Pa against 4.1–10.8 Pa for pressure and 0.75–0.85 $\mathrm{m\,s^{-1}}$ against 1.15–1.29 $\mathrm{m\,s^{-1}}$ for wind. The gap is largest for pressure, whose validation period is the climb to the crater rim, during which the pressure fell below the seasonal baseline (Chapter 4). The twelve boosters train in 37–40 s on the MacBook described in Chapter 5, which reproduced the GBDT metrics of the Windows laptop exactly. LightGBM runs in its deterministic mode. Its row and feature sampling use random seeds of their own, and because the seed of the experiment configuration was not passed to the library, the single runs used the library's default seeds[14]; the seed check of Section 6.7 sets them explicitly to 1 to 5, and the GBDT numbers of Chapter 7 are reported as the mean and standard deviation over these five seeds.

6.5 Temporal convolutional network

6.5.1 Rationale

The TCN is the deep-learning model of the thesis. Whereas the GBDT reads a hand-made summary of the history, the lags and window statistics of Table 6.2, the TCN reads the 24-hour sequence itself and learns which of its patterns matter. Its building block, the causal dilated convolution, looks only backward in time and widens its view geometrically from layer to layer, so that a few layers with few parameters cover a whole sol, and the convolutions of all time steps are computed in parallel, which makes training fast. In an empirical comparison, released as a preprint, TCNs matched or outperformed canonical recurrent networks on most sequence-modeling tasks and retained a longer effective memory[15]. The larger architectures reviewed in Chapter 2, such as DeepAR, N-BEATS, the Temporal Fusion Transformer and other Transformer models, were designed for and evaluated on large collections of related series, and on forecasting benchmarks compact models have repeatedly held their own against them[16]. With one long series per site and a 4 GB GPU as the design constraint (Chapter 1), a compact TCN was therefore chosen; a long short-term memory network[17] was kept as a documented fallback

[14] Cf. *idem*, *Parameters*.

[15] S. Bai, J. Z. Kolter, and V. Koltun. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. 2018. URL: `https://arxiv.org/abs/1803.01271` (accessed 2026-10-07).

[16] Cf. S. Makridakis, E. Spiliotis, and V. Assimakopoulos. "The M4 Competition: 100,000 time series and 61 forecasting methods". In: *International Journal of Forecasting* 36.1 (2020), pp. 54–74; A. Zeng et al. "Are Transformers Effective for Time Series Forecasting?" In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.9 (2023), pp. 11121–11128.

[17] S. Hochreiter and J. Schmidhuber. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780.

and was not needed.

6.5.2 Architecture

Figure 6.2 and Table 6.5 describe the network. The input tensor of 14 channels by 288 steps passes through four residual blocks. Each block applies two causal dilated one-dimensional convolutions with a kernel of three steps, each followed by a rectified linear unit (ReLU) and dropout with a rate of 0.1, and adds the block's input to its output through a skip connection, an identity mapping or a convolution with a kernel of one step where the number of channels changes, followed by a final ReLU. Causality is enforced by trimming the output of each convolution so that it depends only on the current and earlier steps, so that no value after the issue time enters any output. The convolutions are weight-normalized, a reparameterization that separates the length and the direction of each weight vector and speeds up training[18]. Dropout randomly silences units during training and so reduces overfitting[19], and the residual connections let the gradient bypass each block, which eases the training of deep stacks[20]. The blocks have 64, 64, 128 and 128 channels and dilations of 1, 4, 16 and 64 steps. A convolution of kernel k and dilation d widens the receptive field by $(k - 1)d$ steps, so the two convolutions of each block add $4d$ steps and the network sees $1 + 4(1 + 4 + 16 + 64) = 341$ steps, or 28.4 hours, more than the 288-step window: every forecast depends on the whole window. The feature vector at the last time step, of 128 values, is mapped by one linear layer to a scalar, the z-scored forecast. The network has 222,529 parameters, of which 98,816 lie in the last block; the same architecture is trained once per target and horizon.

6.5.3 Hyperparameters and their selection

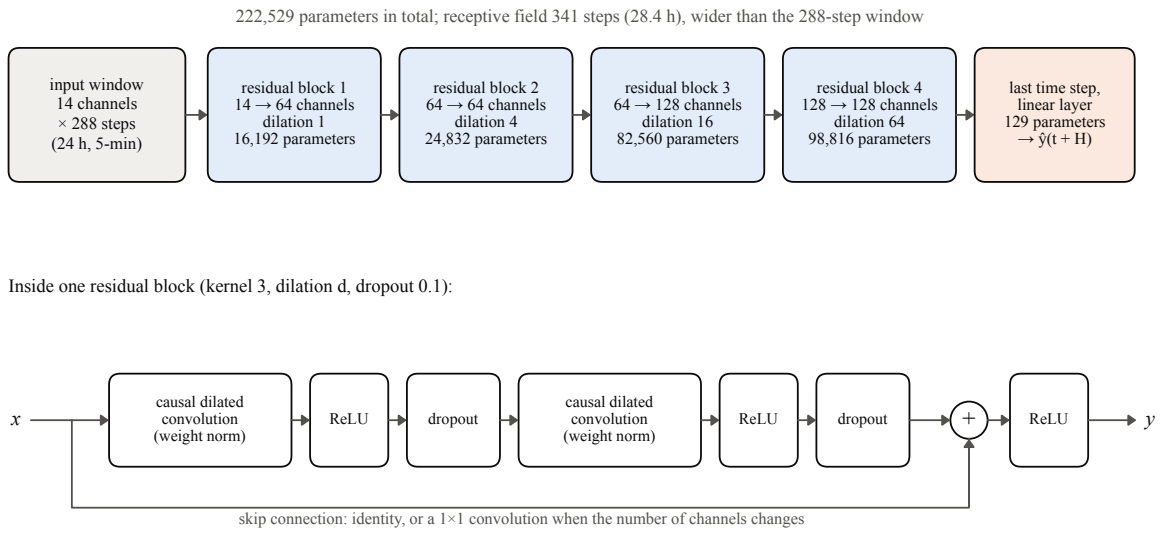
Table 6.6 lists the hyperparameters of the network and of its training. The window of 288 steps, the kernel size, the dilation ladder and the batch size were fixed by the problem and the hardware: the window must cover a sol of history, the dilations must cover the window, and a batch of 128 windows with mixed-precision arithmetic fits the 4 GB of GPU memory (Chapter 5). The remaining settings, the widths of the blocks, the dropout rate, the learning rate and the weight decay, were chosen in a small, structured search on one cell, air temperature at one hour, in which six settings varied one factor at a time around a baseline (Table 6.7). Each setting trained for up to 25 epochs with early stopping after six epochs without improvement, on the training and validation rows of that cell; the search took 19 minutes on the laptop GPU. The selection rule

[18] T. Salimans and D. P. Kingma. "Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 29. Curran Associates, Inc., 2016. URL: https://proceedings.neurips.cc/paper/2016/hash/ed265bc903a5a097f61d3ec064d96d2e-Abstract.html (accessed 2026-11-01).

[19] Cf. N. Srivastava et al. "Dropout: A Simple Way to Prevent Neural Networks from Overfitting". In: *Journal of Machine Learning Research* 15.56 (2014), pp. 1929–1958. URL: https://jmlr.org/papers/v15/srivastava14a.html (accessed 2026-11-01).

[20] Cf. K. He et al. "Deep Residual Learning for Image Recognition". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016, pp. 770–778.

Figure 6.2: Architecture of the temporal convolutional network



Source: own elaboration based on the project code.

Table 6.5: Layers of the TCN, their receptive fields and parameters

Layer	Channels in → out	Dilation	Receptive field (steps)	Parameters
input window	14	—	288	—
residual block 1	14 → 64	1	5	16,192
residual block 2	64 → 64	4	21	24,832
residual block 3	64 → 128	16	85	82,560
residual block 4	128 → 128	64	341	98,816
linear head (last step)	128 → 1	—	—	129
total				222,529

Each block holds two convolutions with a kernel of 3 steps, their weight-normalization parameters, and a 1×1 skip convolution where the channel count changes. The receptive field is counted after the block.

Source: own elaboration based on the project code.

was declared before the search: the setting with the lowest validation RMSE wins, and if the best two lie within 0.5 % of each other, the one with fewer parameters is taken. The setting with a weight decay of 10^{-2} won with 2.041 K; the runner-up, the wide network with four times the parameters, was 1.7 % worse, so the tie-break did not apply. The winning setting was then used for all twelve models, without tuning per target or horizon, and the test period was not touched during the search.

The search must be read with its noise in mind. The six settings span 2.04–2.15 K, about 5 %, whereas the same winning setting, trained again in the final run with the same data and seed, reached 2.170 K on the same validation rows: the initialization differed, because in the search five networks had been trained before the winning setting, whereas the final run began with this

Table 6.6: Hyperparameters and training settings of the TCN

Parameter	Value	Note
input window	288 steps (24 h)	14 channels, z-scored
block widths	64, 64, 128, 128	kept from the search
kernel size, dilations	3; 1, 4, 16, 64	receptive field 341 steps
dropout rate	0.1	after each convolution
optimizer	AdamW	decoupled weight decay
learning rate	10^{-3}	constant, no schedule
weight decay	10^{-2}	winner of the search
batch size	128	windows per step, reshuffled every epoch
loss	MSE of z-scored targets	forecasts transformed back for scoring
gradient clipping	norm 1.0	per step
epochs	up to 50	early stopping after 10 epochs without improvement of the validation RMSE; best epoch kept
arithmetic	mixed precision on CUDA	single precision elsewhere
random seed	42	initialization and shuffling; cuDNN autotuning left on

Source: own elaboration based on the experiment configuration.

Table 6.7: Search over TCN settings on air temperature at a horizon of one hour

Setting	Widths	Dropout	Learning rate	Weight decay	Parameters	Validation RMSE (K)	Best epoch
weight decay	64-64-128-128	0.1	10^{-3}	10^{-2}	222,529	2.041	4
wide	128-128-256-256	0.1	10^{-3}	0	879,233	2.076	4
small	32-32-64-64	0.1	10^{-3}	0	56,993	2.122	4
low learning rate	64-64-128-128	0.1	$5 \cdot 10^{-4}$	0	222,529	2.129	12
baseline	64-64-128-128	0.1	10^{-3}	0	222,529	2.139	4
high dropout	64-64-128-128	0.2	10^{-3}	0	222,529	2.151	4

Up to 25 epochs, early stopping after 6 epochs without improvement; 91,913 training and 22,022 validation rows; Windows laptop GPU, 27 October 2026.

Source: own elaboration based on the search record.

model, and the convolution kernels selected by cuDNN autotuning are not deterministic[21]. The run-to-run variation of about 6 % is thus larger than the spread of the settings, so the search does not establish that one setting is better than another; the declared rule makes the choice reproducible, and that is all that is claimed for it. The same variation bounds what can be read into small differences between the TCN and the GBDT in Chapter 7.

[21] Cf. PyTorch Contributors. *Reproducibility*. PyTorch 2.2 documentation. 2024. URL: `https://docs.pytorch.org/docs/2.2/notes/randomness.html` (accessed 2026-10-28).

6.5.4 Training setup

The network is trained with AdamW, the variant of the Adam optimizer[22] in which the weight decay is applied directly to the weights instead of being added to the gradient[23]. The choice was forced by a defect found in October 2026. With Adam and an L2 penalty coupled into the gradient, the direction vectors of weight-normalized channels that had stopped activating shrank toward zero, to norms of about 10^{-13}, and the gradient of the weight normalization then overflowed after about 2,500 steps. Under mixed precision the loss scaler skipped every such step, so that the weights stopped changing after two to four epochs, which is why earlier training runs had appeared to converge almost immediately; in single precision the network collapsed. In the same probe with decoupled weight decay, the smallest direction norm stayed near 0.5 and no gradient overflowed in 4,000 steps, and an automated test checks that unused weights stay away from zero. All reported runs use the corrected optimizer.

The loss is the MSE of the z-scored target, so that the three targets are trained on one scale; the forecasts are transformed back before any error is computed. The training rows are shuffled every epoch and processed in batches of 128 windows, the gradient norm is clipped at 1.0, and after every epoch the validation RMSE is computed in physical units. Training stops when the validation RMSE has not improved for ten epochs or after 50 epochs, and the weights of the best epoch are restored, the usual form of early stopping[24]. The input and target tensors of a model are built once in memory, up to 121,795 windows of 14 by 288 values in single precision, about 2 GB. On CUDA the batches are copied asynchronously and the forward pass runs in mixed precision; the deterministic convolution algorithms were left off, because they slowed an epoch from about twenty seconds to twenty minutes on the laptop GPU. Training aborts with an error if an epoch ends with a non-finite loss, so that a diverged network cannot pass silently with an early best epoch. The final twelve models were trained on the Windows laptop with the NVIDIA GPU described in Chapter 5, in 31 minutes: an epoch took 12–19 s for air temperature and pressure and 1–2 s for wind, whose training set is an order of magnitude smaller. The network is implemented in PyTorch[25].

Figure 6.3 shows the learning curves of the twelve models. The validation error reaches its minimum at epochs 1 to 6 in eleven of the twelve models and at epoch 19 for pressure at one

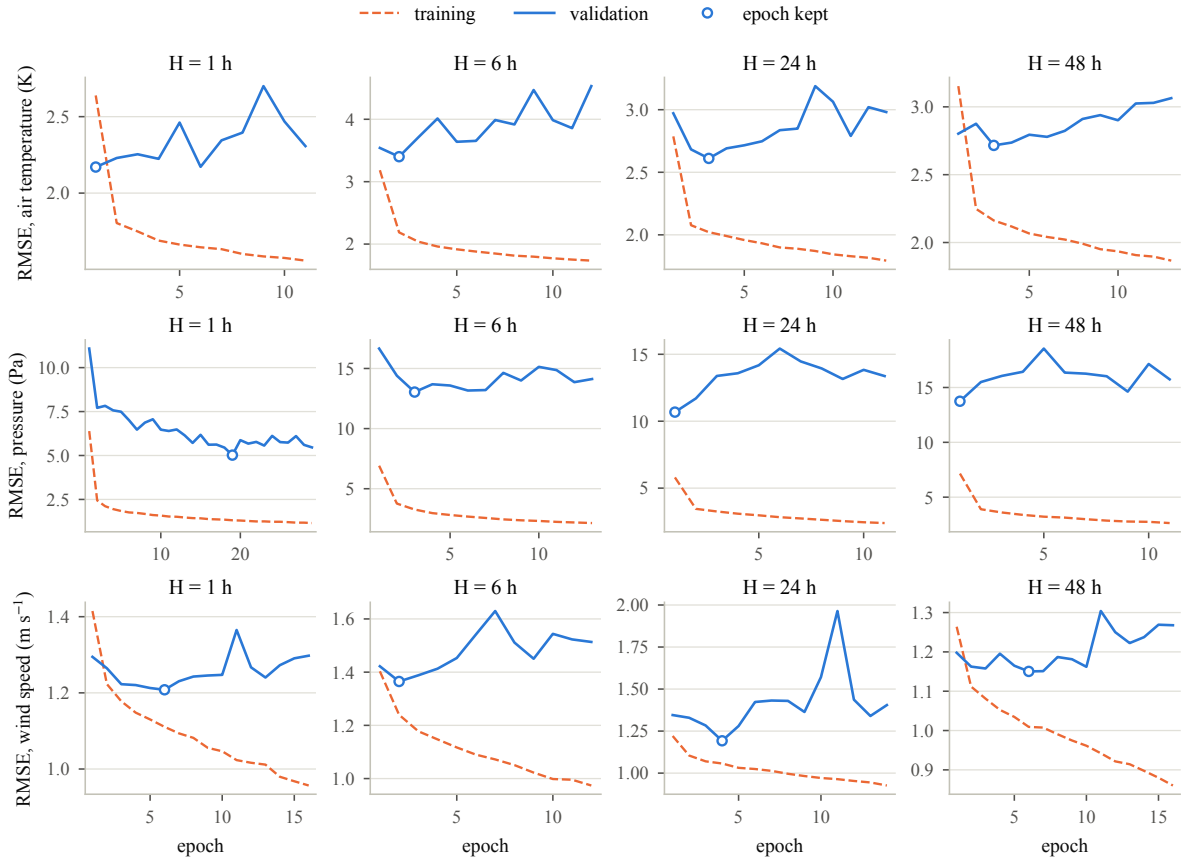
[22] Cf. D. P. Kingma and J. Ba. “Adam: A Method for Stochastic Optimization”. In: *3rd International Conference on Learning Representations (ICLR 2015), Conference Track Proceedings*. San Diego, 2015. URL: https://arxiv.org/abs/1412.6980 (accessed 2026-11-01).

[23] I. Loshchilov and F. Hutter. “Decoupled Weight Decay Regularization”. In: *7th International Conference on Learning Representations (ICLR 2019)*. New Orleans, 2019. URL: https://arxiv.org/abs/1711.05101 (accessed 2026-11-01).

[24] Cf. L. Prechelt. “Early Stopping — But When?” In: *Neural Networks: Tricks of the Trade*. Ed. by G. Montavon, G. B. Orr, and K.-R. Müller. 2nd ed. Lecture Notes in Computer Science 7700. Berlin, Heidelberg: Springer, 2012, pp. 53–67.

[25] A. Paszke et al. “PyTorch: An Imperative Style, High-Performance Deep Learning Library”. In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (accessed 2026-10-07).

Figure 6.3: Training and validation error per epoch of the twelve TCN models



Source: own elaboration based on the learning curves stored with the final TCN run (Chapter 5).

hour, and afterwards it rises in most models, by up to 0.7 K between consecutive epochs for air temperature, while the training error keeps falling. The network therefore fits the training period within a few passes over the data and then overfits it, and early stopping formalizes the choice of the early epoch. The training error of an epoch is the mean over its batches with dropout active, taken while the weights still change, which is why it lies above the validation error in the first epoch of four models. At the kept epoch the validation RMSE is 2.17–3.40 K for air temperature, 5.0–13.7 Pa for pressure and 1.15–1.37 m s^{-1} for wind.

6.6 Design of the cross-site experiments

RQ3 asks whether a model trained at one site generalizes to another. The experiment is a transfer matrix on the hourly grid. For each of the two variables available at all three sites, air temperature and pressure (no REMS wind passes quality control, Chapter 3), and for each horizon, six GBDTs are trained with the settings of Table 6.4: one on the training period of each site, with early stopping on that site's validation period, and one on the pooled training periods of each pair of sites, with early stopping on their pooled validation periods. Each model is then scored on the test period of every site. This gives, per variable and horizon, three in-site cells, six cross-site cells and three pooled cells; the nine cells with a transferred model make 72 in all. The transfer is

zero-shot: no measurement of the test site enters the training or the model selection of a cross-site or pooled model.

The quantity of interest is the transfer gap,

$$G = \frac{\mathrm{RMSE}_{\mathrm{cross}}}{\mathrm{RMSE}_{\mathrm{in}}} - 1, \tag{6.5}$$

the relative increase of the error of a transferred model over the error of the model trained at the test site, computed on identical test rows. Each test period also receives the two reference forecasts of Section 6.3, persistence and the climatology of the change, the latter fitted on the training data of the model under test, so that every cell reports a skill as well. The inputs are the 15 relative features of Table 6.2, formed from the target variable alone, and only complete rows are used. These choices follow from a preliminary version of the experiment, described in Chapter 4, in which the differing duty cycles of the sensors confounded the comparison between sites. Finally, a diagnostic was computed for the interpretation of the gaps: the mean diurnal cycle of each variable at each site over its training period, as the deviation of each hourly value from the mean of its sol, together with the correlation of these cycles and the ratio of their ranges between every pair of sites. Chapter 7 uses it to separate what is shared between the sites, the timing of the cycle, from what is local, its amplitude.

6.7 Evaluation setup

6.7.1 Protocol

Each model is trained once on the training period of its site and target and scored on every issue time of the test period at which the target was measured H hours later: 35,496 issue times for air temperature at one hour, 22,333 for pressure at 48 hours and 1,619 to 2,796 for wind (Table 6.3). This is a dense evaluation with one fixed model: it averages the error over every issue time of about 222 test sols, but it does not re-estimate the model as the test period advances. A rolling-origin evaluation with periodic refits, as recommended in the forecasting literature[26], had been planned and was not implemented; its absence is noted among the limitations in Chapter 7. The test period is used once per model, feature set and seed, and no statistic of the validation or test period enters training (Chapter 4).

All models of an experiment are scored on identical rows. The forecasts of every run are stored with their keys, the target, the horizon and the interval index, and the comparisons join on these keys, so that a paired difference is computed only on issue times scored by both models; the final GBDT and TCN runs share the same 236,127 test rows. For every model, target and horizon, Chapter 7 reports the RMSE, the MAE, the number of rows and the skill against persistence and against the better reference, and for the GBDT the mean and standard deviation over five seeds.

[26] Cf. Tashman, *op. cit.*

6.7.2 Uncertainty of the comparisons

A difference between two forecasts must be judged against the variability of the test period. Forecast errors on Mars are strongly correlated within a sol, so the issue times are not independent observations, and a test that assumes independence would overstate the evidence (Chapter 2). The thesis therefore uses a sol-block bootstrap. Within a cell, the squared errors of the compared forecasts are grouped by sol; the sols of the test period are resampled with replacement 2,000 times, with a fixed seed; in each resample the RMSE of each forecast and the statistic of interest, the difference of RMSEs for RQ2 and the transfer gap and the skill scores for RQ3, are recomputed; and the 2.5th and 97.5th percentiles of the 2,000 values form a 95 % interval. The procedure applies the bootstrap of Efron[27] to blocks of consecutive observations, which preserves the correlation within a block[28]; its blocks are the sols themselves, non-overlapping and of natural length, as in the subseries method of Carlstein[29]. An interval that excludes zero is reported as a significant difference or gap. The intervals describe the sampling of test sols only; they do not cover the variation of training, which the seed check addresses. For the single-run cells of the TCN no interval is given; its run-to-run variation of about 6 % (Section 6.5.3) is reported instead. The Diebold–Mariano test[30], mentioned in Chapter 2, was not used.

6.7.3 Seed check of the paired comparison

A single pair of GBDT runs depends on the random seeds of the row and feature sampling. During the preprocessing corrections of October 2026, the verdict of a single pair of runs on H2 for pressure changed from met to not met and back, the last time after a correction that changed no pressure value, which showed that the sampling of test sols was not the only source of variation. The following protocol was therefore fixed before the corresponding runs. Both feature sets are trained with LightGBM seeds 1 to 5, giving five paired runs. A change is called clear only if all five seed pairs agree in its sign and the sol-block interval of the difference between the seed-averaged forecasts of the two sets excludes zero. H2 is met for a target if at least two horizons show a clear gain of at least 5 % in the mean RMSE. The standard deviation of the RMSE over the five seeds is reported with every GBDT number, and single-run differences smaller than about 5 % of the RMSE, and 10 % for pressure, are treated as noise.

[27] B. Efron. “Bootstrap Methods: Another Look at the Jackknife”. In: *The Annals of Statistics* 7.1 (1979), pp. 1–26.

[28] Cf. H. R. Künsch. “The Jackknife and the Bootstrap for General Stationary Observations”. In: *The Annals of Statistics* 17.3 (1989), pp. 1217–1241.

[29] Cf. E. Carlstein. “The Use of Subseries Values for Estimating the Variance of a General Statistic from a Stationary Sequence”. In: *The Annals of Statistics* 14.3 (1986), pp. 1171–1179.

[30] F. X. Diebold and R. S. Mariano. “Comparing Predictive Accuracy”. In: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263.

6.8 Interpretation and diagnostics

Five diagnostics accompany the metrics in Chapter 7. First, the GBDT exposes the importance of each feature as its share of the total reduction of the training loss achieved by splits on that feature[31]. It is computed after training, on the trained boosters, and never fed back into training. Two cautions apply when reading it: closely related features, such as the air temperature, the temperature of the humidity sensor and the upward longwave flux (Chapter 4), share their importance in proportions that depend on the sampling, and the gain measure favors continuous features with many candidate thresholds over binary flags. Second, the error is decomposed into its bias, the mean of $\hat{y} - y$, and its dispersion, the standard deviation of the same differences, with $\mathrm{RMSE}^2 = \mathrm{bias}^2 + \mathrm{dispersion}^2$; the decomposition separates an offset, such as the shift of pressure with the rover's elevation, from scatter, which call for different remedies. Third, the RMSE is sliced by the LMST hour of the issue time in three-hour bins, which shows where in the sol a model fails, and by quintiles of the test sols, which shows whether a model degrades as the test period moves away from the training period. Fourth, the learning curves of Figure 6.3, the kept rounds of the GBDT and the gaps between training and validation errors serve as the check of overfitting, and the spread of the TCN search and of the GBDT seeds as the measure of stability. Fifth, the diurnal-cycle diagnostic of Section 6.6 interprets the transfer gaps.

The automated tests of Chapter 5 also cover the model code: the alignment and interpolation of the typical day, the scaler and the missing-input handling of the TCN, the behavior of AdamW and the abort on divergence, the period boundaries and the purge of the hourly experiments, and the decision rule of the seed check.

6.9 Summary

The forecasting problem is a direct multi-horizon regression of air temperature, pressure and wind speed 1 to 48 hours ahead from a station's own history, trained with the squared error and judged by the RMSE, the MAE and the skill against persistence and the typical day. Four model classes address it: the two references, a LightGBM ensemble with fixed, conservative settings on 77 or 132 engineered features, and a 222,529-parameter TCN on a 24-hour window of 14 channels, trained with AdamW and early stopping after a small declared search. The three experiments vary one factor each, the model class, the feature set and the training site, over the same chronological periods, the same horizons and identical test rows. Uncertainty is quantified by a sol-block bootstrap for the paired and cross-site comparisons and by five seeds for the GBDT, with decision rules for H1 to H3 fixed before the runs. Chapter 7 reports the results of this protocol.

[31] Cf. Hastie, Tibshirani, and Friedman, *op. cit.*

Bibliography

Bai, S., Kolter, J. Z., and Koltun, V. *An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling*. 2018. URL: https://arxiv.org/abs/1803.01271 (accessed 2026-10-07).

Bergmeir, C. and Benítez, J. M. "On the use of cross-validation for time series predictor evaluation". In: *Information Sciences* 191 (2012), pp. 192–213. DOI: 10.1016/j.ins.2011.12.028.

Carlstein, E. "The Use of Subseries Values for Estimating the Variance of a General Statistic from a Stationary Sequence". In: *The Annals of Statistics* 14.3 (1986), pp. 1171–1179. DOI: 10.1214/aos/1176350057.

Chen, T. and Guestrin, C. "XGBoost: A Scalable Tree Boosting System". In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794. DOI: 10.1145/2939672.2939785.

Diebold, F. X. and Mariano, R. S. "Comparing Predictive Accuracy". In: *Journal of Business & Economic Statistics* 13.3 (1995), pp. 253–263. DOI: 10.1080/07350015.1995.10524599.

Efron, B. "Bootstrap Methods: Another Look at the Jackknife". In: *The Annals of Statistics* 7.1 (1979), pp. 1–26. DOI: 10.1214/aos/1176344552.

Friedman, J. H. "Greedy function approximation: A gradient boosting machine". In: *The Annals of Statistics* 29.5 (2001), pp. 1189–1232. DOI: 10.1214/aos/1013203451.

Grinsztajn, L., Oyallon, E., and Varoquaux, G. "Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?" In: *Advances in Neural Information Processing Systems 35 (Datasets and Benchmarks Track)*. 2022, pp. 507–520. DOI: 10.52202/068431-0037.

Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer Series in Statistics. New York: Springer, 2009. DOI: 10.1007/978-0-387-84858-7.

He, K. et al. "Deep Residual Learning for Image Recognition". In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

Hewamalage, H., Bergmeir, C., and Bandara, K. "Recurrent Neural Networks for Time Series Forecasting: Current status and future directions". In: *International Journal of Forecasting* 37.1 (2021), pp. 388–427. DOI: 10.1016/j.ijforecast.2020.06.008.

Hochreiter, S. and Schmidhuber, J. "Long Short-Term Memory". In: *Neural Computation* 9.8 (1997), pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

Hyndman, R. J. and Athanasopoulos, G. *Forecasting: Principles and Practice*. 3rd ed. Melbourne: OTexts, 2021. URL: https://otexts.com/fpp3/ (accessed 2026-10-07).

Ke, G. et al. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017, pp. 3146–

3154. URL: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html (accessed 2026-10-07).

Kingma, D. P. and Ba, J. "Adam: A Method for Stochastic Optimization". In: *3rd International Conference on Learning Representations (ICLR 2015), Conference Track Proceedings*. San Diego, 2015. URL: https://arxiv.org/abs/1412.6980 (accessed 2026-11-01).

Künsch, H. R. "The Jackknife and the Bootstrap for General Stationary Observations". In: *The Annals of Statistics* 17.3 (1989), pp. 1217–1241. DOI: 10.1214/aos/1176347265.

Loshchilov, I. and Hutter, F. "Decoupled Weight Decay Regularization". In: *7th International Conference on Learning Representations (ICLR 2019)*. New Orleans, 2019. URL: https://arxiv.org/abs/1711.05101 (accessed 2026-11-01).

Makridakis, S., Spiliotis, E., and Assimakopoulos, V. "M5 accuracy competition: Results, findings, and conclusions". In: *International Journal of Forecasting* 38.4 (2022), pp. 1346–1364. DOI: 10.1016/j.ijforecast.2021.11.013.

Makridakis, S., Spiliotis, E., and Assimakopoulos, V. "The M4 Competition: 100,000 time series and 61 forecasting methods". In: *International Journal of Forecasting* 36.1 (2020), pp. 54–74. DOI: 10.1016/j.ijforecast.2019.04.014.

Microsoft Corporation. *Parameters*. LightGBM 4.3.0 documentation. 2024. URL: https://lightgbm.readthedocs.io/en/v4.3.0/Parameters.html (accessed 2026-10-28).

Murphy, A. H. "Climatology, Persistence, and Their Linear Combination as Standards of Reference in Skill Scores". In: *Weather and Forecasting* 7.4 (1992), pp. 692–698. DOI: 10.1175/1520-0434(1992)007<0692:CPATLC>2.0.CO;2.

Murphy, A. H. "Skill Scores Based on the Mean Square Error and Their Relationships to the Correlation Coefficient". In: *Monthly Weather Review* 116.12 (1988), pp. 2417–2424. DOI: 10.1175/1520-0493(1988)116<2417:SSBOTM>2.0.CO;2.

Paszke, A. et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems*. Vol. 32. Curran Associates, Inc., 2019. URL: https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html (accessed 2026-10-07).

Prechelt, L. "Early Stopping — But When?" In: *Neural Networks: Tricks of the Trade*. Ed. by G. Montavon, G. B. Orr, and K.-R. Müller. 2nd ed. Lecture Notes in Computer Science 7700. Berlin, Heidelberg: Springer, 2012, pp. 53–67. DOI: 10.1007/978-3-642-35289-8_5.

PyTorch Contributors. *Reproducibility*. PyTorch 2.2 documentation. 2024. URL: https://docs.pytorch.org/docs/2.2/notes/randomness.html (accessed 2026-10-28).

Salimans, T. and Kingma, D. P. "Weight Normalization: A Simple Reparameterization to Accelerate Training of Deep Neural Networks". In: *Advances in Neural Information Processing Systems*. Vol. 29. Curran Associates, Inc., 2016. URL: https://proceedings.neurips.cc/paper/2016/hash/ed265bc903a5a097f61d3ec064d96d2e-Abstract.html (accessed 2026-11-01).

Srivastava, N. et al. “Dropout: A Simple Way to Prevent Neural Networks from Overfitting”. In: *Journal of Machine Learning Research* 15.56 (2014), pp. 1929–1958. URL: https://jmlr.org/papers/v15/srivastava14a.html (accessed 2026-11-01).

Tashman, L. J. “Out-of-sample tests of forecasting accuracy: an analysis and review”. In: *International Journal of Forecasting* 16.4 (2000), pp. 437–450. DOI: 10.1016/S0169-2070(00)00065-0.

Zeng, A. et al. “Are Transformers Effective for Time Series Forecasting?” In: *Proceedings of the AAAI Conference on Artificial Intelligence* 37.9 (2023), pp. 11121–11128. DOI: 10.1609/aaai.v37i9.26317.